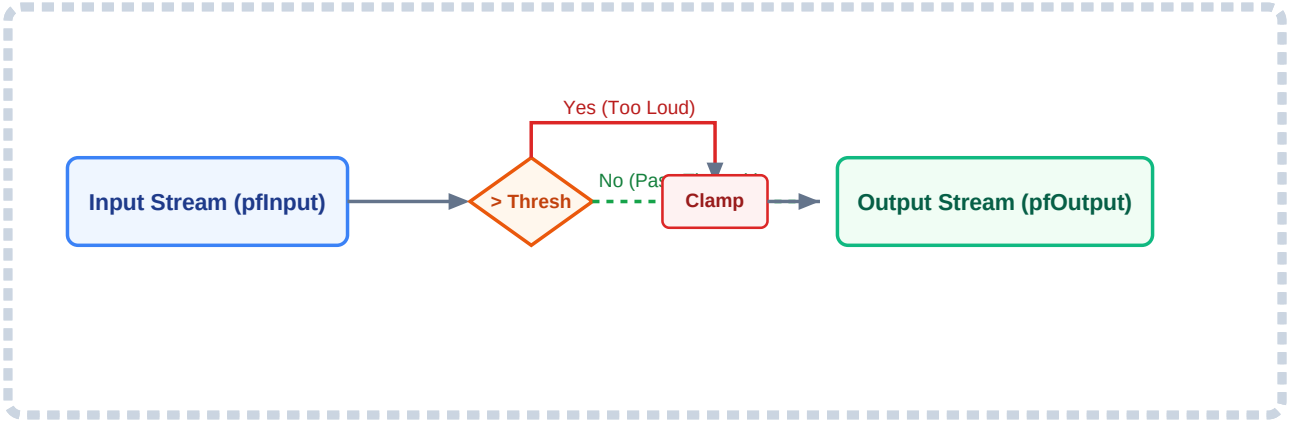
Audio Module Documentation: Limiter Block

A simple breakdown of how the limiter protects our audio system from clipping and distortion.

1. High-Level Diagram

This diagram shows how audio data moves through the limiter loop. The module checks every single incoming audio sample one-by-one. If a sample stays below our safety limit, it passes right through. If it tries to cross the limit, the code instantly clamps it down.



2. Module Structures & Memory Size

To keep our code clean, we separate the user's settings from the active workspace handle that stays in the memory.

Configuration Structure (`limiter_config_t`)

This is used just at the beginning in our initialization function to pass settings from our main code into the limiter module.

- **`float fThreshold`** : This is our maximum volume ceiling (usually a value between `0.0f` and `1.0f`). Any audio wave peak trying to go higher than this number will get cut off to keep speakers from popping or distorting.

Workspace Handle Structure (`limiter_hdl_t`)

This acts as the active instance of our limiter in memory. It stores our configuration parameters locally so the module can keep running smoothly even after the startup code finishes execution.

- **`float fThreshold`** : The internal copy of our volume ceiling value.

Memory Usage Table

Variable Name	Data Type	Memory Size	Dynamic Allocation (Malloc)
`phdl->fThreshold`	Standard 32-bit Float	4 Bytes	0 Bytes (Statically Handled)

3. Simple Mathematical Logic

The limiter behaves like an automatic safety wall. It watches the incoming wave instantly without looking at past history.

The simple rule our processing loop follows is:

> If the sound wave sample is between -Thresh and +Thresh → Leave it alone.
> If the sound wave sample goes above +Thresh or below -Thresh → Chop it down to exactly the Thresh value.

Where:

- **Incoming Sample**: The raw real-time live microphone data coming into our system.
- **Outbound Sample**: The safe, hard-bounded data sent out to our speaker.

- **Thresh Value**: The local threshold boundary ceiling we configured in our handle.

4. Lifecycle Functions & Loop Execution

The code is divided into clear step-by-step lifecycle phases.
we do not simply assume an API is safe for in-place processing by default.
Compatibility boundaries depend entirely on the design of the execution loop; therefore, this memory behavior boundary contract must be explicitly documented directly in the public header file.

A. Query Memory Size (`limiter_open`)

```
STATUS limiter_open(uint32_t *pui32Size) {
    if (pui32Size == NULL) return STATUS_NOT_OK;
    *pui32Size = sizeof(limiter_hdl_t);
    return STATUS_OK;
}
```

What it does: Tells our main program exactly how many bytes of memory are needed to house this limiter instance. This keeps our internal structural variables abstract and hidden from the outer framework loops.

B. Initialize Module Settings (`limiter_init`)

```
STATUS limiter_init(limiter_hdl_t *phdl, const limiter_config_t *psConfig) {
    if (phdl == NULL || psConfig == NULL) return STATUS_NOT_OK;
    phdl->fThreshold = psConfig->fThreshold;
    return STATUS_OK;
}
```

What it does: Checks our memory references, reads the chosen threshold boundary from the configuration struct, and saves it permanently inside our module instance handle.

C. Process Audio Samples (`limiter_process`)

```
/**
 * @brief Restricts output stream samples within specified ceiling thresholds.
```

```
 * @note [IN-PLACE COMPATIBLE] This loop reads each input sample into a local
 * register variable before writing back out. It is completely safe to pass the
 * exact same buffer pointer for both pfInput and pfOutput.
 */
STATUS limiter_process(limiter_hdl_t *phdl, const float *pfInput, float *pfOutp
    if (phdl == NULL || pfInput == NULL || pfOutput == NULL) return STATUS_NOT_O

    for (uint32_t i = 0; i < ui32NumSamples; i++) {
        float sample = pfInput[i];

        if (sample > phdl->fThreshold) {
            sample = phdl->fThreshold;
        } else if (sample < -phdl->fThreshold) {
            sample = -phdl->fThreshold;
        }

        pfOutput[i] = sample;
    }
    return STATUS_OK;
}
```

What it does: Runs a rapid loop through our current chunk of audio data. Notice that it safely copies the raw input value into a local CPU register (`float sample`) before modifying any memory blocks. Because it reads before it writes, it is “in-place compatible”, meaning you can safely use the same tracking array for both input and output without risk of corrupted data.

D. Close and Clean Up (`limiter_close`)

```
STATUS limiter_close(limiter_hdl_t *phdl) {
    if (phdl == NULL) return STATUS_NOT_OK;
    return STATUS_OK;
}
```

What it does: Closes out our module gracefully once our application shuts down. Since all of our handles are statically declared across our pipeline scope, no complex heap cleanup loops or `free()` calls are required here.